ByteMart Digital

THE COMPLETE GUIDE

Manual Testing

A Complete Guide to Software Quality

A complete, practical guide to manual software testing — from the STLC and test design to defect management, specialized testing, and professional QA practice.

30	80+	10	8
Chapters	Examples	Visual Diagrams	Modules

© 2026 ByteMart Digital · All Rights Reserved

ByteMart Digital

Manual Testing: The Complete Guide

First Edition · 2026

Copyright © 2026 ByteMart Digital. All rights reserved.
No part of this publication may be reproduced, distributed, or transmitted in any form or by any means — including photocopying, recording, screenshotting, or other electronic or mechanical methods — without the prior written permission of the publisher, except for brief quotations used in reviews and certain other non-commercial uses permitted by copyright law.

This product is licensed for personal use by the purchaser only. Sharing, reselling, or redistributing this file in whole or in part is strictly prohibited and constitutes a violation of copyright law.

Disclaimer
The information in this book is provided in good faith and for educational purposes. While every effort has been made to ensure accuracy, the author and publisher assume no responsibility for errors, omissions, or outcomes arising from the use of the code or information contained herein. All code examples are provided "as is".

Trademarks
Python is a registered trademark of the Python Software Foundation. All other product names, logos, and brands are the property of their respective owners and are used for identification purposes only.

Published by ByteMart Digital

Web: bytemartdigital.com
Email: contact@bytemartdigital.in
Instagram: instagram.com/bytemartdigital.in
Telegram: t.me/bytemartcodingcommunity

Contents

1. What Is Software Testing? 4
2. Why Testing Matters 6
3. SDLC & the V-Model 8
4. The Seven Testing Principles 11
5. The Software Testing Life Cycle 13
6. Verification & Validation 16
7. Levels of Testing 19
8. The Testing Process Overview 22
9. Functional vs Non-Functional Testing 24
10. Black, White & Grey Box Testing 27
11. Smoke, Sanity & Regression Testing 30
12. Static vs Dynamic Testing 32
13. Test Case Design 34
14. Equivalence Partitioning & BVA 37
15. Decision Tables & State Transition 40
16. Exploratory & Ad-hoc Testing 42
17. Requirement Analysis & Traceability 44
18. Test Planning 46
19. Writing Effective Test Cases 48
20. Test Execution & Reporting 50
21. The Defect Life Cycle 52
22. Severity vs Priority 55
23. Bug Reporting Best Practices 57
24. Usability & UI Testing 60
25. Compatibility & Cross-Browser Testing 62
26. Mobile Application Testing 64
27. API Testing (Manual / Postman) 66
28. Testing in Agile 69
29. Test Metrics & Management Tools 71
30. QA Best Practices & Career 73

MODULE 1 · TESTING FOUNDATIONS

CHAPTER 01

What Is Software Testing?

The purpose of testing — finding defects and building confidence in software quality.

Software testing is the process of evaluating an application to find defects and verify that it meets its requirements. It is how teams build confidence that software works correctly, reliably, and as users expect — before it reaches those users.

Software Testing
The process of executing a program or system with the intent of finding defects, and of verifying that it satisfies specified requirements and works as expected.

Testing vs debugging

Testing finds defects — it reveals that something is wrong. **Debugging** is what developers do afterward to locate and fix the cause. Testers report the symptom; developers cure the disease.

What testing is really about

- **Finding defects** before customers do.
- **Verifying requirements** are met.
- **Building confidence** in a release.
- **Reducing risk** of costly failures in production.

Manual vs automated testing

Manual testing is performed by a human tester who executes test cases without automation tools — essential for exploratory, usability, and ad-hoc testing. **Automated testing** uses scripts and is ideal for repetitive, regression-heavy checks. This guide focuses on manual testing, the foundation every tester must master first.

Testing shows presence, not absence
Testing can prove that defects exist, but never that a program is completely defect-free. The goal is to reduce risk to an acceptable level — not to achieve the impossible ideal of perfection.

TIP
Great testers think like curious, slightly pessimistic users — constantly asking 'what if?' and 'what could go wrong here?' rather than only confirming the happy path works.

© 2026 ByteMart Digital

MODULE 1 · TESTING FOUNDATIONS

CHAPTER 02

Why Testing Matters

The cost of defects, and why quality is cheaper the earlier you find problems.

Bugs are expensive — and they get dramatically more expensive the later they're found. Understanding the economics of quality is what turns testing from a chore into a business-critical activity.

The cost of defects rises over time

Found during	Relative cost to fix
Requirements	1x
Design	~5x
Development / testing	~10x
Production (after release)	~30–100x

A defect caught in requirements is a quick edit; the same defect discovered by customers can mean emergency fixes, lost revenue, and damaged trust. This is why early testing pays off so handsomely.

The real cost of failure

- **Financial** — lost sales, refunds, emergency fixes.
- **Reputation** — users lose trust and switch to competitors.
- **Safety** — in medical, automotive, or finance, bugs can be catastrophic.
- **Legal** — compliance failures and liability.

Quality assurance vs quality control

Quality Assurance (QA) is process-oriented and preventive — building quality in. **Quality Control (QC)** is product-oriented and detective — finding defects in the built product. Testing is a QC activity within a broader QA process.

Shift left

The earlier testing starts, the cheaper defects are to fix. 'Shifting left' — involving testers from the requirements stage onward — is one of the highest-leverage practices in software quality.

TIP

When advocating for testing time, frame it in money and risk, not just 'quality'. 'This bug in production could cost us thousands' lands better than 'we need more testing'.

© 2026 ByteMart Digital

MODULE 1 · TESTING FOUNDATIONS

CHAPTER 03

SDLC & the V-Model

Where testing fits in the software development life cycle.

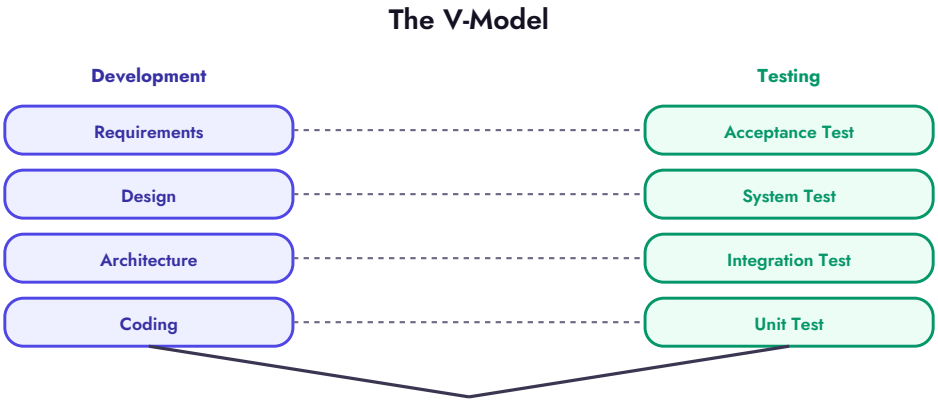
Testing doesn't happen in isolation — it's woven throughout the Software Development Life Cycle (SDLC). The V-Model makes this explicit by pairing every development phase with a corresponding testing phase.

The SDLC phases

The **SDLC** describes how software is built: requirements, design, development, testing, deployment, and maintenance. Different models — Waterfall, V-Model, Agile — arrange these phases differently.

The V-Model

The **V-Model** extends Waterfall by mapping each development activity on the left to a testing activity on the right. Test planning starts early, in parallel with development — not as an afterthought.



Each development phase (left) has a matching testing phase (right).

Development phase	Testing phase
Requirements	Acceptance testing
High-level design	System testing
Detailed design	Integration testing
Coding	Unit testing

Waterfall vs Agile testing

- **Waterfall** — sequential; testing is a distinct late phase.
- **V-Model** — testing planned alongside each dev phase.
- **Agile** — continuous testing every short iteration (sprint).

© 2026 ByteMart Digital

Test early, test throughout

The V-Model's core lesson is that test design should begin as soon as requirements exist. Waiting until code is written to think about testing is a recipe for missed defects and rushed cycles.

TIP

Even in fast-moving Agile teams, the V-Model mindset holds — for every feature, ask 'how will we test this?' while it's still being designed, not after it's built.

© 2026 ByteMart Digital

MODULE 1 · TESTING FOUNDATIONS

CHAPTER 04

The Seven Testing Principles

Timeless truths that guide effective, efficient testing.

Decades of testing experience distilled into seven principles. They explain what testing can and cannot achieve, and they steer testers away from common traps and toward high-impact effort.

The principles

- **1. Testing shows the presence of defects** — never their complete absence.
- **2. Exhaustive testing is impossible** — you can't test every input and path.
- **3. Early testing saves time and money** — shift left.
- **4. Defects cluster together** — a few modules hold most bugs (the Pareto effect).
- **5. Beware the pesticide paradox** — the same tests stop finding new bugs; vary them.
- **6. Testing is context-dependent** — a banking app and a game need different testing.
- **7. Absence-of-errors is a fallacy** — a bug-free product that doesn't meet needs is still a failure.

Why they matter

These principles keep expectations realistic and effort focused. Knowing exhaustive testing is impossible pushes you toward smart test design; knowing defects cluster tells you where to concentrate.

The pesticide paradox in practice

Running the identical test suite over and over eventually catches nothing new — like a pesticide insects grow immune to. The fix is to regularly review and add new test cases, and to explore untested areas.

Principles over rote steps
Good testers internalise these principles and apply judgement, rather than mechanically following a script. The principles explain the why behind every technique in this guide.

TIP
Principle 7 is the one people forget: software that passes every test but doesn't actually solve the user's problem is not quality software. Always tie testing back to real user needs.

© 2026 ByteMart Digital

MODULE 2 · THE TESTING LIFE CYCLE

CHAPTER 05

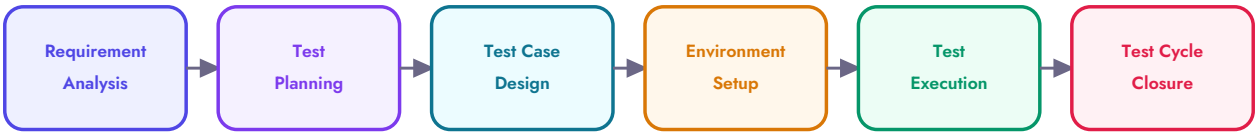
The Software Testing Life Cycle

The STLC — the structured phases every testing effort moves through.

Just as software has a development life cycle, testing has its own: the Software Testing Life Cycle (STLC). It gives testing a clear, repeatable structure — from understanding requirements to closing the test cycle.

The six phases

The Software Testing Life Cycle (STLC)



Testing flows through six structured phases, each with its own entry and exit criteria.

Phase	Key activity
Requirement analysis	understand what to test (RTM)
Test planning	strategy, scope, effort, resources
Test case design	write cases and prepare test data
Environment setup	prepare the test environment
Test execution	run cases, log defects
Cycle closure	report, review, lessons learned

Entry and exit criteria

Each phase has **entry criteria** (what must be ready to start) and **exit criteria** (what must be complete to move on). For example, test execution can't begin until the environment is set up and test cases are reviewed.

Deliverables

- **Requirement analysis** → Requirements Traceability Matrix.
- **Planning** → test plan and effort estimate.
- **Design** → test cases and test data.
- **Closure** → test summary report.

© 2026 ByteMart Digital

Structure prevents chaos

The STLC ensures nothing is skipped — that requirements are understood before cases are written, and cases are ready before execution starts. It turns testing from ad-hoc effort into a disciplined process.

TIP

Clear exit criteria are your protection against endless testing and against shipping too early. Agree them upfront: 'we're done when X% of cases pass and no critical defects remain open.'

© 2026 ByteMart Digital

MODULE 2 · THE TESTING LIFE CYCLE

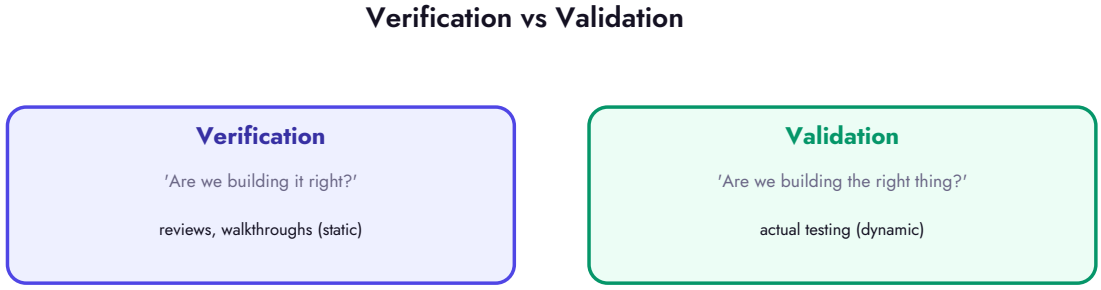
CHAPTER 06

Verification & Validation

Two complementary checks — building it right, and building the right thing.

'Verification' and 'validation' sound similar but mean different things, and interviewers love the distinction. Together they answer two essential questions about any product.

The two questions



Verification checks we build it right; validation checks we build the right thing.

Verification	Validation
'building it right?'	'building the right thing?'
static (no execution)	dynamic (running the software)
reviews, walkthroughs, inspections	actual testing
checks documents & design	checks the working product

Verification techniques

- **Reviews** — examining documents and code for issues.
- **Walkthroughs** — the author guides peers through the work.
- **Inspections** — formal, checklist-driven examination.

Validation techniques

Validation means actually executing the software — functional testing, system testing, user acceptance testing — to confirm it meets the user's real needs, not just the written spec.

Both are essential

You can build a product perfectly to spec (verification passes) and still build the wrong product (validation fails). Quality requires both — correct construction and the right goal.

TIP

A memory hook: **V**erification = a**V**oid defects early via reviews; **V**alidation = **V**erify with the real running product. Both start with V, but they happen at different times and in different ways.

MODULE 2 · THE TESTING LIFE CYCLE

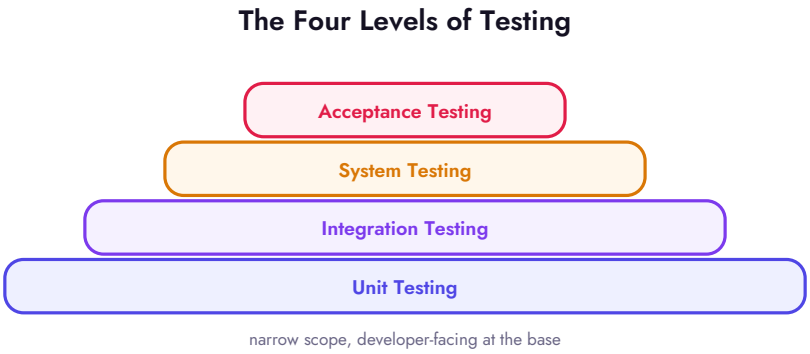
CHAPTER 07

Levels of Testing

Testing at increasing scope — from a single unit to the whole system.

Software is tested at multiple levels, each covering a wider scope than the last. Moving from unit to acceptance testing catches different kinds of defects, so every level plays a distinct, necessary role.

The four levels



Testing widens in scope from individual units up to user acceptance.

Level	Tests	Done by
Unit	individual components	developers
Integration	combined modules	developers / testers
System	the complete system	testers
Acceptance	against user needs	users / clients

Integration approaches

- **Big Bang** — integrate everything at once, then test.
- **Top-down** — test high-level modules first, using stubs.
- **Bottom-up** — test low-level modules first, using drivers.
- **Sandwich** — combine top-down and bottom-up.

User acceptance testing

UAT is the final level, where real users confirm the software meets their needs in realistic scenarios. Its variants include Alpha testing (in-house) and Beta testing (by real users in the field).

Each level catches different bugs
Unit tests catch logic errors in isolation; integration tests catch interface mismatches; system tests catch end-to-end issues; acceptance tests catch 'this isn't what we wanted.' Skipping a level leaves a gap.

© 2026 ByteMart Digital

TIP

As a manual tester, you'll spend most of your time at the system and acceptance levels — testing complete workflows the way a real user would experience them.

© 2026 ByteMart Digital

MODULE 2 · THE TESTING LIFE CYCLE

CHAPTER 08

The Testing Process Overview

How the pieces fit — a bird's-eye view of testing from plan to sign-off.

Before diving into individual techniques, it helps to see the whole testing process end to end. This chapter connects the concepts so far into a single coherent workflow you'll return to throughout your career.

The end-to-end flow

- **1. Understand** — analyse requirements and clarify ambiguities.
- **2. Plan** — define scope, approach, and what 'done' means.
- **3. Design** — write test cases and prepare data.
- **4. Execute** — run cases, compare actual vs expected.
- **5. Report** — log defects and communicate status.
- **6. Close** — summarise results and capture lessons.

Actual vs expected

At its core, every test compares an **expected result** (from the requirements) against the **actual result** (what the software does). A mismatch is a potential defect — which is why clear, correct expected results are so important.

Roles in the process

Role	Responsibility
Test Lead / Manager	planning, strategy, reporting
Test Engineer	design and execute test cases
Developer	fix defects, unit test
Business Analyst	clarify requirements

It's a communication job
Testing is as much about clear communication — with developers, analysts, and managers — as it is about finding bugs. A defect no one understands or acts on delivers no value.

TIP
Keep a running note of assumptions and questions as you test. Ambiguous requirements are a leading source of both defects and wasted effort — surfacing them early saves everyone time.

© 2026 ByteMart Digital

MODULE 3 · TESTING TYPES

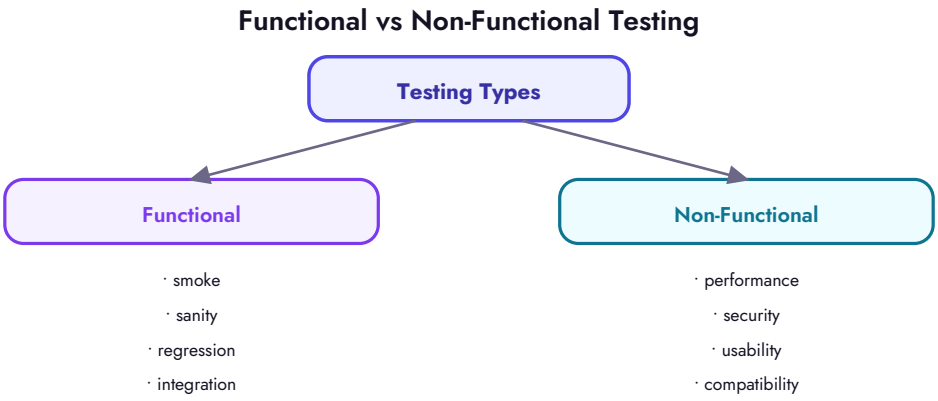
CHAPTER 09

Functional vs Non-Functional Testing

Testing what the software does versus how well it does it.

All testing splits into two broad families. **Functional** testing checks what the system does; **non-functional** testing checks how well it does it. A complete test effort needs both.

The two families



Functional testing verifies behaviour; non-functional testing verifies qualities.

Functional testing

Functional testing verifies that features behave according to requirements — does login work, does the cart calculate totals correctly, does the form validate input? It's based on the specification, not the code.

- Smoke, sanity, and regression testing.
- Integration and system testing.
- User acceptance testing.

Non-functional testing

Type	Checks
Performance	speed, responsiveness under load
Security	protection against threats
Usability	ease of use
Compatibility	works across devices/browsers
Reliability	stability over time

Both or neither

A feature that works perfectly but takes 30 seconds to load, or leaks user data, has still failed the user. Functional correctness and non-functional quality are equally part of 'working' software.

© 2026 ByteMart Digital

TIP

As a manual tester you'll focus mostly on functional and usability testing; performance and security often use specialised tools — but knowing they exist and when to flag them is part of the job.

© 2026 ByteMart Digital

MODULE 3 · TESTING TYPES

CHAPTER 10

Black, White & Grey Box Testing

How much you know about the internals shapes how you test.

Testing techniques are often classified by how much the tester knows about the internal code. This ranges from knowing nothing (black box) to full visibility (white box), with a middle ground between.

The three approaches



Black box tests behaviour; white box tests internals; grey box sits between.

Approach	Knowledge of code	Focus
Black box	none	inputs & outputs
Grey box	partial	both, informed
White box	full	internal logic & paths

Black box testing

The tester treats the system as a sealed box — feeding inputs and checking outputs against expectations, without seeing the code. Most manual functional testing is black box. Techniques include equivalence partitioning and boundary value analysis.

White box testing

The tester examines the internal code and structure, designing tests to exercise specific paths, branches, and statements. It's usually done by developers and measured with **code coverage**.

Grey box testing

A blend: the tester has partial internal knowledge — say, the database schema or architecture — and uses it to design smarter black-box tests. Common in integration and security testing.

© 2026 ByteMart Digital

Match the technique to the goal

Black box mirrors the user's perspective and catches requirement gaps; white box ensures internal thoroughness. The best test strategies combine perspectives rather than relying on one.

TIP

Even in pure black-box roles, a little grey-box knowledge — how data flows, where the boundaries are — makes you far more effective at spotting where bugs are likely to hide.

© 2026 ByteMart Digital

MODULE 3 · TESTING TYPES

CHAPTER 11

Smoke, Sanity & Regression Testing

Three everyday functional test types that keep releases stable.

These three test types come up constantly in real projects and interviews. Each answers a different question about a new build — and confusing them is a classic mistake.

Smoke testing

Smoke testing is a quick, broad check that the critical functions of a new build work at all — 'does it even turn on?' If smoke tests fail, the build is rejected before deeper testing begins.

Sanity testing

Sanity testing is a narrow, focused check that a specific fix or feature works after a minor change — verifying the reasoning is 'sane' before proceeding. It's usually unscripted.

Smoke	Sanity
broad & shallow	narrow & deep
whole build stability	a specific area/fix
often scripted	usually unscripted

Regression testing

Regression testing re-runs existing tests after a change to confirm that new code hasn't broken previously working features. It's the safety net of every release — and the area where automation pays off most.

WATCH OUT

Regression is where bugs love to hide: a fix in one place breaks something seemingly unrelated. Never assume a 'small change' is safe without regression testing the affected areas.

Different questions, different tests

Smoke asks 'is the build stable enough to test?'; sanity asks 'does this specific fix work?'; regression asks 'did we break anything that used to work?' Knowing which to run when is core testing judgement.

TIP

Maintain a prioritised regression suite covering your most critical user journeys. When time is short, running that focused suite catches the highest-impact breakages fast.

© 2026 ByteMart Digital

MODULE 3 · TESTING TYPES

CHAPTER 12

Static vs Dynamic Testing

Testing without running the code, and testing by running it.

Not all testing involves executing the software. **Static** testing examines artefacts without running them; **dynamic** testing runs the code. Both catch different kinds of defects, and the cheapest bugs are caught statically.

Static testing

Static testing reviews requirements, designs, and code without execution — through reviews, walkthroughs, inspections, and static-analysis tools. It catches defects early, when they're cheapest to fix.

- **Reviews** of requirements and design documents.
- **Code reviews** and pair programming.
- **Static analysis** tools that scan code for issues.

Dynamic testing

Dynamic testing executes the software with inputs and observes behaviour — everything from unit tests to manual system testing. It's what most people picture when they think 'testing.'

Static	Dynamic
no execution	code is executed
reviews, inspections	running test cases
finds defects early	finds runtime defects
examines documents & code	examines behaviour

Cheapest defects are static
A defect caught in a requirements review never becomes code, never ships, and never needs a fix-and-retest cycle. Static testing is the ultimate 'shift left' — don't overlook it in favour of only running tests.

TIP
Participate actively in requirement and design reviews. Questions you raise there — 'what happens if this field is empty?' — prevent defects that would otherwise cost far more to catch later.

MODULE 4 · TEST DESIGN TECHNIQUES

CHAPTER 13

Test Case Design

Anatomy of a good test case — clear, repeatable, and traceable.

A test case is a documented set of steps, inputs, and expected results used to verify a feature. Well-designed test cases are the backbone of manual testing — clear enough for anyone to execute and get the same result.

Anatomy of a test case

Anatomy of a Test Case

Test Case ID	`TC_LOGIN_01`
Precondition	`user is registered`
Test Steps	`1. open login  2. enter creds  3. submit`
Test Data	`user@x.com / pass123`
Expected Result	`dashboard is shown`
Actual Result	`dashboard is shown`
Status	`Pass`

A complete test case leaves no ambiguity about what to do or what to expect.

Field	Purpose
Test Case ID	unique identifier
Precondition	state required before running
Test Steps	exact actions to perform
Test Data	inputs to use
Expected Result	what should happen
Actual Result	what did happen
Status	Pass / Fail

Characteristics of a good test case

- **Clear** — unambiguous steps anyone can follow.
- **Atomic** — tests one thing, so failures are precise.
- **Repeatable** — same steps give the same result.
- **Traceable** — links back to a requirement.

Positive and negative cases

© 2026 ByteMart Digital

Positive cases verify the system works with valid input (login with correct credentials). **Negative** cases verify it handles invalid input gracefully (login with a wrong password shows a clear error). Good coverage needs both.

Write for a stranger

A test case should be executable by someone who has never seen the feature. If it needs your verbal explanation to run, it isn't finished. That clarity is what makes tests repeatable and shareable.

TIP

Don't forget the 'unhappy paths' — empty fields, wrong types, huge inputs, special characters. Negative test cases catch more real defects than confirming the happy path ever will.

© 2026 ByteMart Digital

MODULE 4 · TEST DESIGN TECHNIQUES

CHAPTER 14

Equivalence Partitioning & BVA

Two techniques to test smart — fewer cases, better coverage.

Since exhaustive testing is impossible, we need techniques that pick the most valuable inputs. Equivalence partitioning and boundary value analysis do exactly that — dramatically cutting the number of tests while keeping coverage high.

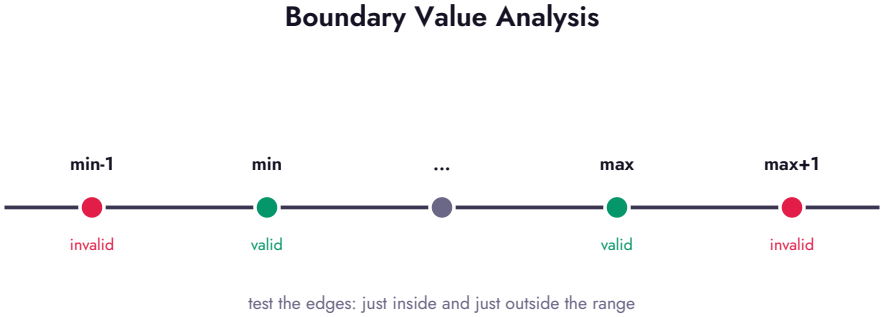
Equivalence partitioning

Equivalence Partitioning (EP) groups inputs into classes that should behave the same way, then tests just one representative from each class. If a field accepts 1–100, then 50 represents all valid values — no need to test 49, 51, 52...

```text
Field accepts age 18-60:
  Invalid class:  < 18   -> test 10
  Valid class:    18-60  -> test 35
  Invalid class:  > 60   -> test 70
=> 3 tests instead of dozens
```

Boundary value analysis

Boundary Value Analysis (BVA) targets the edges of each partition — where defects cluster. For a range, test the minimum, maximum, and the values just inside and outside them.



Bugs hide at the edges — test just inside and just outside every boundary.

```text
Range 18-60, boundary values to test:
  17 (invalid), 18 (valid min), 19,
  59, 60 (valid max), 61 (invalid)
```

Why they work together

EP reduces the total number of test cases; BVA ensures the risky edges within those partitions are covered. Used together, they give strong coverage with a small, defensible set of tests.

Test where bugs live

Off-by-one errors and wrong comparison operators cluster at boundaries — exactly what BVA targets. These two techniques turn 'test everything' into 'test the few inputs that actually matter.'

TIP

Whenever you see a range, a limit, or a length restriction, reach for BVA immediately. Boundaries are the single most productive place to look for defects.

© 2026 ByteMart Digital

MODULE 4 · TEST DESIGN TECHNIQUES

CHAPTER 15

Decision Tables & State Transition

Techniques for combinations of conditions and state-driven behaviour.

Some features depend on combinations of conditions, others on the sequence of events. Decision tables and state transition testing are the techniques designed for exactly these situations.

Decision table testing

A **decision table** maps combinations of input conditions to expected actions — ideal when behaviour depends on several conditions together, like eligibility or pricing rules.

Member?	Cart > 500?	Discount
Yes	Yes	20%
Yes	No	10%
No	Yes	5%
No	No	0%

Each row is a test case. The table guarantees you cover every meaningful combination rather than guessing which to try.

State transition testing

State transition testing checks behaviour as the system moves between states in response to events — like an order going draft → placed → shipped → delivered. You verify valid transitions work and invalid ones are blocked.

```text
Order states:
  draft -> placed   (on submit)   valid
  placed -> shipped (on dispatch) valid
  delivered -> placed             invalid, must be blocked
```

Cover combinations and sequences

Decision tables ensure no combination of conditions is missed; state transition testing ensures no illegal sequence slips through. Both replace guesswork with systematic coverage.

TIP

Decision tables are also a fantastic way to clarify requirements — building one often reveals combinations the specification never mentioned, surfacing gaps before any code is tested.

MODULE 4 · TEST DESIGN TECHNIQUES

CHAPTER 16

Exploratory & Ad-hoc Testing

Unscripted testing that relies on tester skill, intuition, and curiosity.

Not all valuable testing follows a script. Exploratory and ad-hoc testing let skilled testers investigate the application freely — often uncovering defects that rigid test cases would never reach.

Exploratory testing

Exploratory testing is simultaneous learning, test design, and execution — the tester explores the application, forms hypotheses, and follows leads in real time. It's structured but not scripted, and highly effective.

Ad-hoc testing

Ad-hoc testing is informal, unstructured testing without documentation — trying random inputs and unusual flows to 'break' the software. It relies purely on tester experience and creativity.

Scripted	Exploratory
pre-written cases	designed on the fly
repeatable, documented	flexible, investigative
good for coverage & regression	good for finding the unexpected

When to use them

- New or rapidly changing features with thin documentation.
- After scripted testing, to probe beyond the obvious.
- Under time pressure, to find high-impact defects fast.
- When requirements are vague and you're learning as you go.

Scripts and exploration together

Scripted tests give coverage and repeatability; exploratory testing finds the surprising, high-value defects hiding in the gaps. The strongest testers use both, not one or the other.

TIP

Time-box exploratory sessions and take notes as you go (a 'test charter'). This keeps free-form testing focused and makes any defects you find reproducible for developers.

© 2026 ByteMart Digital

MODULE 5 · THE TESTING WORKFLOW

CHAPTER 17

Requirement Analysis & Traceability

Understanding what to test, and proving every requirement is covered.

Testing starts long before execution — with truly understanding the requirements. Ambiguous or misunderstood requirements are a leading cause of both defects and wasted effort, so this first step is critical.

Analysing requirements

The tester reads the requirements, identifies what is testable, and — crucially — flags anything ambiguous, incomplete, or contradictory. A question asked here saves a defect later.

- Is each requirement **clear** and unambiguous?
- Is it **testable** — can you write a pass/fail check?
- Is it **complete** — are edge cases and errors specified?
- Is it **consistent** with the other requirements?

The Requirements Traceability Matrix

An **RTM** maps each requirement to the test cases that verify it. It proves coverage — every requirement has at least one test — and reveals gaps where requirements are untested or tests have no requirement.

Requirement	Test Cases	Status
REQ-01 Login	TC-01, TC-02, TC-03	Covered
REQ-02 Reset password	TC-04, TC-05	Covered
REQ-03 Logout	—	Gap!

Trace to prove coverage

The RTM answers the question every manager eventually asks: 'how do we know we've tested everything?' It makes coverage visible and gaps impossible to hide.

TIP

Testable requirements are specific and measurable. 'The page should load fast' is untestable; 'the page should load within 2 seconds' is. Push for the latter during requirement reviews.

© 2026 ByteMart Digital

MODULE 5 · THE TESTING WORKFLOW

CHAPTER 18

Test Planning

The strategy document that guides the entire testing effort.

A test plan is the blueprint for testing a project — defining what will be tested, how, by whom, and when. A good plan aligns everyone and sets clear expectations before a single test runs.

What a test plan contains

- **Scope** — what's in and out of scope.
- **Approach** — the testing strategy and types.
- **Resources** — people, tools, environments.
- **Schedule** — timelines and milestones.
- **Entry / exit criteria** — when to start and stop.
- **Risks** — what could go wrong and the mitigation.

Entry and exit criteria

Entry criteria define what must be ready before testing begins (stable build, environment, test cases). **Exit criteria** define when testing is complete (e.g. 95% of cases passed, no open critical defects).

Estimating effort

Estimation predicts how long testing will take — based on the number of test cases, complexity, and past data. Realistic estimates protect quality from being squeezed by unrealistic deadlines.

WATCH OUT

Testing time is the first thing cut when a project runs late. A clear test plan with agreed exit criteria is your best defence — it makes the quality trade-off of skipping tests explicit and visible.

Plan the work, work the plan

The test plan turns vague intentions into concrete commitments. It's where you negotiate scope and time before the pressure hits — not in the chaos of a late release.

TIP

Keep the plan living, not shelf-ware. When scope or timelines change, update the plan and re-confirm exit criteria — an outdated plan is worse than none.

© 2026 ByteMart Digital

MODULE 5 · THE TESTING WORKFLOW

CHAPTER 19

Writing Effective Test Cases

Turning requirements into clear, thorough, maintainable test cases.

Writing test cases is the craft at the heart of manual testing. This chapter turns the design techniques you've learned into practical, well-structured cases that catch defects and stay useful over time.

From requirement to cases

For each requirement, derive positive cases (valid input works), negative cases (invalid input is handled), and boundary cases (edges behave correctly). One requirement typically yields several test cases.

```text
REQ: Password must be 8-20 chars
  TC1  8 chars   -> accepted   (boundary, valid)
  TC2  20 chars  -> accepted   (boundary, valid)
  TC3  7 chars   -> rejected   (boundary, invalid)
  TC4  21 chars  -> rejected   (boundary, invalid)
  TC5  empty     -> rejected   (negative)
```

Common mistakes to avoid

- Vague steps that assume knowledge ('test the login').
- Combining many checks into one case, so failures are unclear.
- Missing expected results — without them, there's no pass/fail.
- Only testing the happy path.

Keeping test cases maintainable

Software changes, so test cases must be easy to update. Keep them modular, avoid duplicating steps, reference shared preconditions, and review them regularly to prune the obsolete.

One case, one purpose
A test case that checks a single behaviour tells you exactly what broke when it fails. Cramming multiple checks into one case saves writing time but costs you clarity when it matters most.

TIP
Reuse a library of common steps and preconditions ('user is logged in'). It keeps cases short, consistent, and far easier to update when the workflow changes.

© 2026 ByteMart Digital

MODULE 5 · THE TESTING WORKFLOW

CHAPTER 20

Test Execution & Reporting

Running test cases, recording results, and communicating status.

Execution is where plans meet reality — running test cases, comparing actual against expected, and recording what happens. Clear reporting turns those results into decisions the whole team can act on.

The execution cycle

- Run each test case step by step.
- Compare the **actual result** to the **expected result**.
- Mark **Pass** or **Fail**; log a defect for each failure.
- Retest fixed defects; run regression as needed.

Recording results

Every executed case gets a clear status. Failures need enough detail — steps, data, environment, and evidence like screenshots — for a developer to reproduce the problem without guessing.

The test summary report

Metric	Example
Cases executed	240 / 250
Passed / Failed	228 / 12
Defects open	4 (1 critical)
Pass rate	95%

Communicating status

A good status report tells stakeholders what they need to decide: is the product ready to ship? It highlights blockers and risks clearly — not just raw numbers, but what those numbers mean for the release.

Results drive decisions

Test execution produces the evidence a go/no-go decision rests on. Your job isn't just to run cases — it's to translate results into a clear, honest picture of release readiness.

TIP

Attach concrete evidence to every failure — screenshots, logs, exact steps. 'It doesn't work' wastes a developer's day; 'these 3 steps produce this error, see attached' gets it fixed fast.

© 2026 ByteMart Digital

MODULE 6 · DEFECT MANAGEMENT

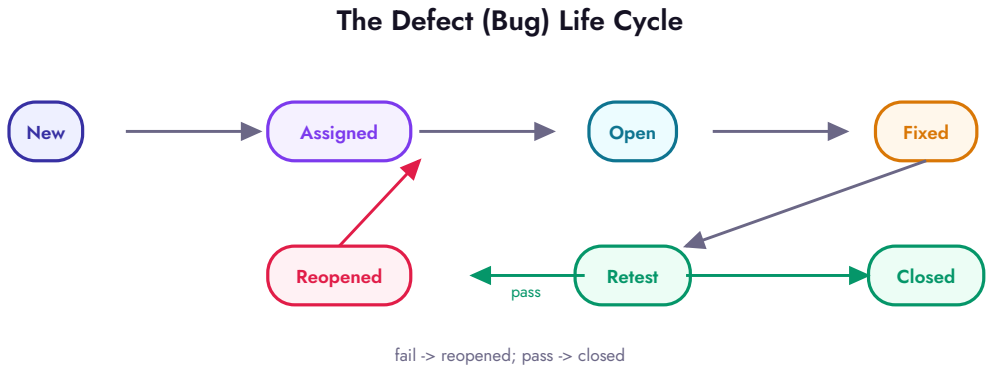
CHAPTER 21

The Defect Life Cycle

How a bug travels from discovery to closure.

When a tester finds a defect, it doesn't just get fixed and forgotten — it moves through a defined life cycle with clear states. Understanding this flow keeps everyone aligned on each bug's status.

The states a defect passes through



A defect moves through defined states from New to Closed — or back to Reopened.

State	Meaning
New	just logged by the tester
Assigned	given to a developer
Open	developer is working on it
Fixed	developer has resolved it
Retest	tester re-verifies the fix
Closed	confirmed fixed
Reopened	still failing — back to the developer

Other resolutions

- **Duplicate** — already reported elsewhere.
- **Rejected** — not a valid defect.
- **Deferred** — valid, but fixed in a later release.
- **Not a bug** — the software works as designed.

Retesting vs regression

Retesting confirms a specific fixed defect is now resolved. **Regression testing** checks that the fix didn't break anything else. Both happen after a fix — they're complementary, not the same.

© 2026 ByteMart Digital

Track every defect to closure

A defect life cycle prevents bugs from falling through the cracks. Every reported defect ends in a definitive state — fixed, rejected, or deferred — with nothing left in limbo.

TIP

Always retest a fix using the exact steps from the original report before closing it. 'Should be fixed' is not 'confirmed fixed' — reopened bugs erode trust fast.

© 2026 ByteMart Digital

MODULE 6 · DEFECT MANAGEMENT

CHAPTER 22

Severity vs Priority

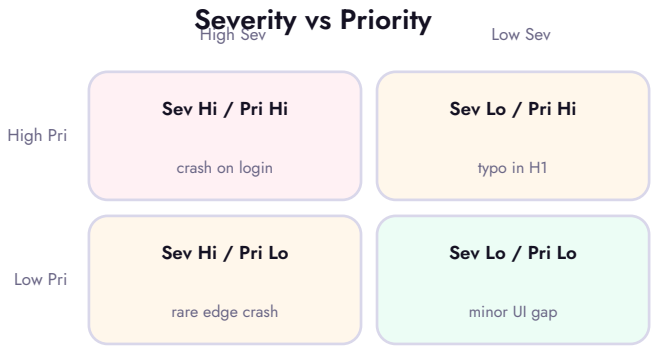
Two independent measures that decide which bugs get fixed first.

'Severity' and 'priority' are the most confused pair of terms in testing — and a favourite interview question. They measure different things, and a bug can be high in one and low in the other.

The two measures

- **Severity** — the technical impact of the defect on the system. Set by the tester.
- **Priority** — the business urgency to fix it. Set by the product owner / manager.

The four combinations



Severity is technical impact; priority is business urgency — they're independent.

Combination	Example
High severity, high priority	app crashes on login
High severity, low priority	crash in a rarely used feature
Low severity, high priority	typo in the company logo
Low severity, low priority	minor spacing on an admin page

The logo typo example is the classic case: technically trivial (low severity) but embarrassing and public, so it must be fixed immediately (high priority).

Impact vs urgency
Severity asks 'how badly does it break the system?'; priority asks 'how soon must we fix it for the business?' Separating them lets teams triage sensibly — not every severe bug is urgent, and not every urgent bug is severe.

TIP
As the tester, you own severity — base it on technical impact, objectively. Leave priority to the business, but flag anything customer-facing or reputational; those often deserve high priority regardless of severity.

MODULE 6 · DEFECT MANAGEMENT

CHAPTER 23

Bug Reporting Best Practices

Writing defect reports that get bugs understood, reproduced, and fixed.

A defect is only as useful as the report that describes it. A vague bug report wastes everyone's time; a clear one gets the bug fixed fast. Writing great reports is a core, career-defining tester skill.

Anatomy of a great bug report

text

```
Title:    Checkout fails when cart total is exactly 1000
Severity: High     Priority: High     Env: Chrome 120, Staging

Steps to reproduce:
  1. Add items totalling exactly 1000 to the cart
  2. Click 'Checkout'

Expected: Order proceeds to payment
Actual:   Page shows 'Invalid amount' error

Attachments: screenshot.png, console-log.txt
```

What makes it work

- **Clear title** — summarises the problem at a glance.
- **Exact steps** — anyone can reproduce it.
- **Expected vs actual** — shows precisely what's wrong.
- **Environment** — browser, OS, build, data.
- **Evidence** — screenshots, logs, videos.

The cardinal rule: reproducibility

If a developer can't reproduce a defect, they can't fix it. The single most valuable thing a bug report provides is clear, reliable reproduction steps — everything else supports that goal.

WATCH OUT

Avoid emotional or vague language ('the checkout is completely broken!'). Stick to precise, factual, reproducible observations — they get bugs fixed; complaints get them ignored.

A bug report is a fix request

Your report is the interface between finding a bug and getting it fixed. Invest in making it clear and reproducible — it's the difference between a same-day fix and a week of back-and-forth.

© 2026 ByteMart Digital

TIP

Before logging, try to reproduce the bug a second time yourself and find the minimal steps that trigger it. A tight, reliable repro is worth more than pages of description.

© 2026 ByteMart Digital

MODULE 7 · SPECIALIZED TESTING

CHAPTER 24

Usability & UI Testing

Testing whether software is easy, pleasant, and intuitive to use.

Software can be functionally perfect yet frustrating to use. Usability and UI testing evaluate the human experience — whether users can accomplish their goals easily, and whether the interface looks and behaves correctly.

UI testing

UI testing checks the visual and interactive elements: layout, alignment, colours, fonts, buttons, and responsiveness. Does everything display correctly and respond as expected across screen sizes?

- Layout, alignment, and spacing are consistent.
- Text is free of typos and displays fully.
- Buttons, links, and forms work and give feedback.
- The interface adapts to different screen sizes.

Usability testing

Usability testing goes deeper — evaluating how easy and intuitive the software is. Can a new user complete key tasks without confusion? Are error messages helpful? Is navigation logical?

Accessibility

Accessibility testing ensures the software is usable by people with disabilities — keyboard navigation, screen-reader support, sufficient colour contrast, and alt text. It follows standards like WCAG, and is increasingly a legal requirement.

> **Test for the human, not just the code**
> Functional correctness is necessary but not sufficient. If real users struggle, get confused, or can't access your product, it has failed them — usability and accessibility are part of quality.

> **TIP**
> The strongest usability insight comes from watching a real, first-time user attempt a task in silence. The moments they hesitate or misclick reveal problems no checklist would surface.

MODULE 7 · SPECIALIZED TESTING

CHAPTER 25

Compatibility & Cross-Browser Testing

Ensuring software works across browsers, devices, and platforms.

Users access software from countless combinations of browsers, devices, operating systems, and screen sizes. Compatibility testing verifies the application works consistently across the ones that matter to your audience.

Dimensions of compatibility

Dimension	Examples
Browsers	Chrome, Firefox, Safari, Edge
Operating systems	Windows, macOS, Android, iOS
Devices	desktop, tablet, phone
Screen sizes	4K down to small mobile
Networks	fast wifi to slow 3G

Cross-browser testing

Browsers render pages differently and support features unevenly. **Cross-browser testing** confirms the layout, functionality, and performance hold up across the browsers your users actually use.

Prioritising combinations

Testing every possible combination is impossible. Use your **analytics** to focus on the browsers, devices, and OS versions your real users have — covering the top combinations that represent most of your traffic.

WATCH OUT

Don't test only on your own machine and browser. 'Works on my machine' is a classic trap — a layout that's perfect in your Chrome may be broken in Safari or on a small phone screen.

Meet users where they are

Compatibility testing is about the real diversity of your audience, not a theoretical matrix. Data-driven prioritisation gives you strong coverage without testing endless combinations that no one uses.

TIP

Cloud testing platforms (BrowserStack, LambdaTest) let you check many browser/device combinations without owning the hardware — invaluable when your users span a wide range of devices.

© 2026 ByteMart Digital

MODULE 7 · SPECIALIZED TESTING

CHAPTER 26

Mobile Application Testing

The unique challenges of testing apps on phones and tablets.

Mobile apps introduce challenges desktop software never faces — fragmented devices, interruptions, gestures, and constrained resources. Mobile testing addresses this distinctive environment.

What makes mobile different

- **Device fragmentation** — countless models, sizes, and OS versions.
- **Interruptions** — calls, notifications, low battery mid-use.
- **Network variability** — wifi, 5G, 3G, and offline.
- **Gestures & sensors** — tap, swipe, pinch, rotation, GPS.
- **Resources** — limited memory, battery, and storage.

Native, web, and hybrid apps

Type	Built with
Native	platform SDK (Swift, Kotlin)
Web	runs in the mobile browser
Hybrid	web wrapped in a native shell

Key mobile test scenarios

Beyond functionality, test how the app handles interruptions, rotation, backgrounding and resuming, low battery, poor connectivity, and different permissions — the real-world conditions phones face constantly.

Test real-world conditions

A mobile app must survive an interrupting phone call, a rotated screen, and a dropped signal — conditions that never occur on a desktop. Testing only the happy path in perfect conditions misses the defects users hit daily.

TIP

Test on real devices for the important cases, not just emulators. Emulators miss real-world factors like actual performance, touch responsiveness, battery behaviour, and genuine network conditions.

© 2026 ByteMart Digital

MODULE 7 · SPECIALIZED TESTING

CHAPTER 27

API Testing (Manual / Postman)

Testing the interfaces behind the UI — directly and thoroughly.

Behind most applications are APIs — the interfaces that move data between systems. Testing them directly catches defects earlier and more thoroughly than testing only through the UI. Postman is the go-to manual tool.

Why test APIs directly

- Faster and more stable than UI testing.
- Catches defects before the UI is even built.
- Tests logic, validation, and error handling directly.
- Verifies security and data integrity at the source.

The anatomy of an API request

```http
POST /api/users
Host: api.example.com
Content-Type: application/json

{ "name": "Sam", "email": "sam@x.com" }
```

What to check in the response

Check	Example
Status code	200 OK, 201 Created, 404, 500
Response body	correct data, structure, types
Headers	content-type, auth, caching
Response time	within acceptable limits

Common HTTP status codes

2xx means success, 3xx redirection, 4xx a client error (like 400 bad request or 401 unauthorised), and 5xx a server error. Verifying the right code is returned — including for invalid input — is core API testing.

Test the source of truth

The API is where the real logic and data live. Testing it directly finds defects the UI would hide, and confirms the system behaves correctly for both valid and invalid requests — including proper error codes.

© 2026 ByteMart Digital

TIP

Always test the negative cases: missing fields, wrong types, invalid auth, and oversized payloads. A robust API returns clear, correct error responses — don't just confirm the happy path returns 200.

MODULE 8 · PROFESSIONAL TESTING

CHAPTER 28

Testing in Agile

How testing works in fast, iterative Agile teams.

Most modern teams work in Agile, where software is built in short iterations and testing is continuous rather than a final phase. This changes how, when, and by whom testing is done.

Agile vs traditional testing

Traditional	Agile
testing is a late phase	testing every sprint
separate QA team	testers embedded in the team
heavy documentation	lightweight, just-enough docs
test after build	test continuously

The tester's role in a sprint

- Clarify acceptance criteria during planning.
- Test features as they're completed, not at the end.
- Collaborate closely with developers and the product owner.
- Automate regression so it keeps pace with rapid change.

Key Agile concepts

User stories capture requirements from the user's view; **acceptance criteria** define when a story is 'done'; the **definition of done** usually includes 'tested'. Testers help shape all three.

Quality is everyone's job

In Agile, testing isn't a gate at the end — it's a continuous, shared responsibility woven into every sprint. The tester's job shifts from gatekeeper to collaborator who builds quality in from the start.

TIP

Get involved at the start of each story, not the end. Reviewing acceptance criteria during planning catches ambiguity and untestable requirements while they're still cheap to fix.

© 2026 ByteMart Digital

MODULE 8 · PROFESSIONAL TESTING

CHAPTER 29

Test Metrics & Management Tools

Measuring testing progress and managing it with the right tools.

Testing generates data — and the right metrics turn that data into insight about progress, quality, and risk. Management tools organise the whole effort, from requirements to defects to reports.

Useful test metrics

Metric	Tells you
Test case pass rate	how much is working
Defect density	defects per module / size
Defect leakage	bugs that reached production
Test coverage	how much is tested
Defect resolution time	how fast bugs get fixed

Reading metrics wisely

Metrics guide decisions but can mislead if worshipped. A high pass rate means little if coverage is low; counting bugs found rewards quantity over impact. Use metrics to ask better questions, not to game targets.

Test management tools

- **Jira** — issue and defect tracking, Agile boards.
- **TestRail / Zephyr** — organising test cases and runs.
- **Confluence** — test plans and documentation.
- **Bugzilla** — dedicated defect tracking.

WATCH OUT

Don't let a metric become the goal. If testers are judged purely on 'bugs found', they'll log trivial issues; if on 'pass rate', they'll write easy tests. Measure to learn, not to rank people.

Measure to improve

Good metrics reveal where quality risks lie and whether the process is working. The aim is insight and better decisions — not a dashboard that looks green while defects slip into production.

TIP

'Defect leakage' — bugs that escaped to production — is one of the most honest quality metrics. A rising trend is a clear signal your testing needs strengthening in specific areas.

© 2026 ByteMart Digital

MODULE 8 · PROFESSIONAL TESTING

CHAPTER 30

QA Best Practices & Career

Habits of great testers, and how to grow a testing career.

Becoming an excellent tester is about mindset and habits as much as techniques. This final chapter gathers the practices that distinguish great QA professionals — and maps the paths a testing career can take.

Habits of great testers

- **Think like a user**, and like an adversary trying to break things.
- **Question assumptions** — in requirements and in your own tests.
- **Communicate clearly** — precise, factual, actionable reports.
- **Prioritise ruthlessly** — focus effort where risk is highest.
- **Keep learning** — tools, domains, and the product evolve.

The tester's mindset

The best testers are curious, detail-oriented, and constructively skeptical. They take pride not in software being broken, but in preventing users from ever experiencing those breaks. It's a quality-advocate role, not a fault-finder role.

Career paths in QA

Path	Focus
Manual → Automation	learn to code test scripts
Test Lead / Manager	strategy, planning, people
Specialist	performance, security, mobile
SDET	developer + tester hybrid

Where to go next

Manual testing is the essential foundation — the mindset and techniques here underpin everything, including automation. A natural next step is learning to automate the repetitive checks so you can focus your human judgement where it matters most.

You are the user's advocate

A tester's real job is to protect users from bad experiences and protect the business from costly failures. Master the fundamentals in this guide, keep that advocacy mindset, and you'll bring value to any team.

© 2026 ByteMart Digital

TIP

The strongest career move for a manual tester is learning automation — not to replace your judgement, but to free it from repetitive work. The companion Automation Testing guide is the natural next step.

© 2026 ByteMart Digital